

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Y. Ramadevi	Reg. No.:	192373065
Date:	10/08/2026	Duration:	60 minutes

Code of Conduct

*I **Y. Ramadevi - 192373065** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Y. Ramadevi

Annexure A – Code Review & Repository Evaluation

Scalable Online Food Ordering System

Develop a scalable full-stack Online Food Ordering System that allows customers to browse restaurants, view menus, search and filter food items, add items to a cart, place orders, and track order status through a web-based application.

The system shall use React for the frontend and Node.js with Express.js for backend REST APIs. A database such as MongoDB shall be used to store customer details, restaurant information, menu items, cart data, orders, and delivery status.

The system shall provide separate functionalities for customers, restaurant administrators, and system administrators. Customers can browse restaurants and place orders, restaurant administrators can manage menus and orders, and administrators can monitor users, restaurants, and overall system activity.

The application shall be developed using Git and GitHub for version control and collaborative development. The frontend and backend shall be containerized using Docker, while Docker Compose can be used to manage the frontend, backend, and database services.

The system should be designed with scalability, maintainability, performance, security, and reliability in mind.

1. Problem Overview

1.1 Problem Description

Traditional food ordering processes can involve multiple steps such as browsing menus, contacting restaurants, placing orders, and tracking order status. A centralized online food ordering platform can simplify these activities by providing a single digital system.

The proposed Scalable Online Food Ordering System provides customers with an easy-to-use platform for discovering restaurants, selecting food items, placing orders, and tracking their orders.

The system also provides restaurant administrators with facilities to manage menus and orders.

The application follows a full-stack architecture consisting of:

- React frontend
- Node.js/Express backend
- REST APIs
- MongoDB database
- Git/GitHub version control
- Docker containerization
- Docker Compose for multi-container execution

1.2 Project Objectives

The major objectives are:

1. To develop a user-friendly online food ordering platform.
2. To allow customers to browse restaurants and food menus.
3. To provide food searching, filtering, and sorting facilities.
4. To allow customers to add food items to a shopping cart.
5. To provide online order placement and order tracking.
6. To allow restaurants to manage menus and incoming orders.
7. To provide administrators with system monitoring capabilities.
8. To implement RESTful APIs for frontend-backend communication.
9. To manage the project using Git and GitHub.
10. To containerize the application using Docker.
11. To design the application for scalability and maintainability.

1.3 Key Functionalities

Customer Module

- User registration and login
- Restaurant browsing
- Food menu viewing
- Search food items
- Filter food by category
- Sort food by price or rating
- Add items to cart
- Update cart quantities
- Place orders
- View order history
- Track order status

Restaurant Module

- Restaurant login
- Add food items
- Update food items
- Delete food items
- Manage food categories
- View incoming orders

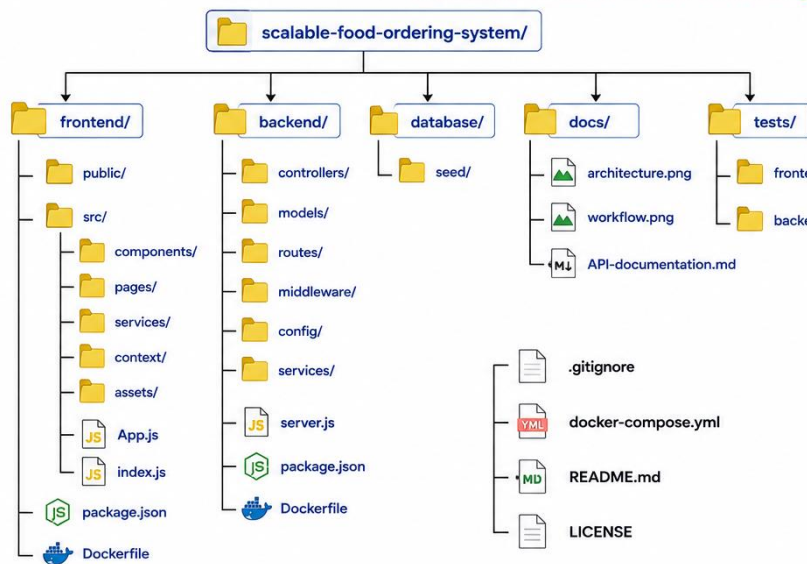
- Update order status
- Monitor restaurant sales

Administrator Module

- Manage users
- Manage restaurants
- Monitor orders
- View system statistics
- Manage application information

2. Repository Organization

Scalable Online Food Ordering System



Item	Description / Purpose
frontend/	Contains React frontend application (UI of the food ordering system)
backend/	Contains Node.js / Express backend source code and APIs
database/	Contains database initialization scripts and seed data
docs/	Contains project documentation, architecture diagram, workflow, and API reference
tests/	Contains test cases for frontend and backend
.gitignore	Specifies files and folders to be ignored by Git
docker-compose.yml	Defines and manages multi-container Docker services
README.md	Project overview, setup instructions, usage guide, and other information
LICENSE	Project license information

Key Highlights

- ✓ Well-organized structure for better maintainability.
- ✓ Separation of frontend, backend, database, documentation and tests.
- ✓ Supports collaborative development and version control.
- ✓ Docker files for containerization and easy deployment.

Benefits of this Structure

- ✓ Easy navigation and file management.
- ✓ Encourages modular and clean coding practices.
- ✓ Simplifies testing, documentation, and deployment.
- ✓ Improves collaboration in team development.

2.1 Purpose of Major Folders

Folder/File	Purpose
frontend/	Contains the React application
components/	Contains reusable UI components
pages/	Contains application pages
services/	Contains API communication functions
context/	Manages shared application state
assets/	Stores images and static resources
backend/	Contains Node.js/Express application
controllers/	Contains application business logic
models/	Defines database schemas
routes/	Defines REST API endpoints

middleware/	Authentication and request-processing functions
config/	Database and application configuration
services/	Contains backend service logic
database/	Database initialization and sample data
docs/	Project documentation and diagrams
tests/	Test cases
.gitignore	Specifies files not tracked by Git
Dockerfile	Defines container image creation
docker-compose.yml	Defines multiple application services
README.md	Provides project setup and usage information

2.2 Repository Organization Evaluation

The repository follows a modular structure where frontend, backend, database, documentation, and testing resources are separated.

This organization improves:

- Code readability
- Maintainability
- Collaboration
- Debugging
- Testing
- Feature development
- Version control

Clear folder and file names allow developers to quickly locate the required source code.

3. Code Quality Review

3.1 Code Readability

The source code should follow consistent indentation, meaningful variable names, proper spacing, and appropriate comments.

Example:

```
const calculateTotal = (items) => {
  return items.reduce(
    (total, item) => total + item.price * item.quantity,
    0
  );
};
```

The function name calculateTotal clearly describes its purpose.

3.2 Modularity

The application should be divided into independent modules.

For example:

Restaurant Module



Menu Module



Cart Module



Order Module



Payment Module



Delivery Module

Each module performs a specific responsibility.

This reduces code duplication and makes the application easier to maintain.

3.3 Coding Standards

The following practices should be followed:

- Use meaningful variable names.
- Use reusable functions.
- Avoid duplicate code.
- Keep functions small.
- Use consistent indentation.
- Use comments where required.
- Handle errors properly.
- Validate user input.
- Store sensitive information in environment variables.
- Avoid hard-coded passwords and API keys.

3.4 Code Quality Strengths

The proposed application demonstrates:

- Modular source code
- Separation of frontend and backend

- Reusable React components
- REST API structure
- Database models
- Error handling
- Input validation
- Containerized deployment
- Version-controlled source code

3.5 Areas for Improvement

Possible improvements include:

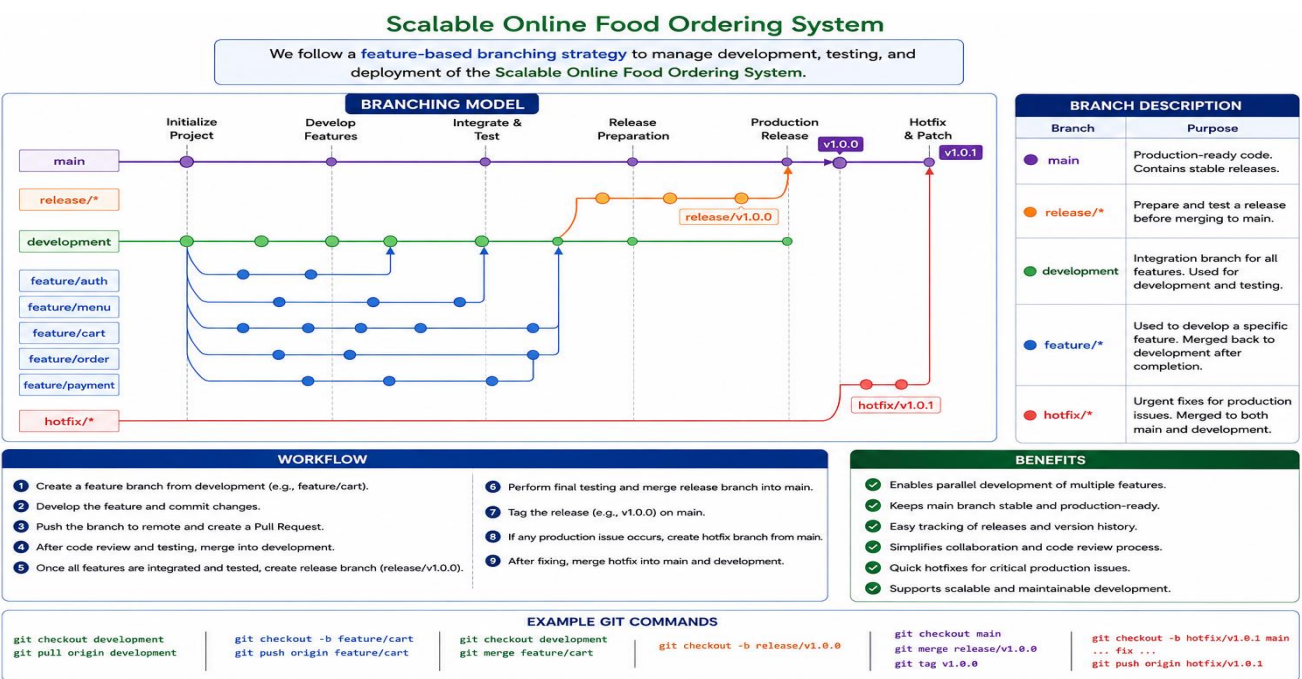
- Increase automated test coverage.
- Improve error messages.
- Add stronger authentication.
- Optimize database queries.
- Implement caching.
- Improve API documentation.
- Add centralized logging.
- Introduce continuous integration and deployment.

4. Version Control Evaluation

Git and GitHub are used to manage the source code and support collaborative development.

4.1 Git Branching Strategy

The project can use the following branches:



Main Branch

Contains stable production-ready code.

Development Branch

Contains integrated development work.

Feature Branches

Used to develop individual features.

Examples:

feature/user-authentication

feature/restaurant-menu

feature/shopping-cart

feature/order-management

feature/payment

feature/admin-dashboard

Release Branch

Used for final testing and release preparation.

Hotfix Branch

Used to resolve critical production issues.

4.2 Commit History Evaluation

Example commit history:

Initial project setup

Create React frontend

Add Express backend

Add restaurant model

Implement menu management

Add food search functionality

Implement shopping cart

Add order management API

Add order tracking

Add Docker configuration

Update README documentation

Fix cart calculation issue

Improve API validation

These commit messages are meaningful because they clearly describe the changes introduced by each commit.

4.3 Version Control Benefits

Git provides:

- Change tracking
- Branch management
- Collaboration
- Code review
- Conflict resolution
- Rollback
- Complete development history

GitHub additionally provides:

- Remote repository
- Pull requests
- Code reviews
- Issue tracking
- Team collaboration

5. Code Testing and Validation

Testing is performed to ensure that all major system functions work correctly.

5.1 Test Cases

Test ID	Test Case	Expected Result
TC01	User registration	User account created successfully
TC02	User login	User successfully authenticated
TC03	View restaurants	Restaurant list displayed
TC04	Search food item	Matching food items displayed
TC05	Filter menu	Correct category displayed
TC06	Add food to cart	Item added successfully
TC07	Update cart quantity	Cart total updated
TC08	Place order	Order created successfully
TC09	View order history	Previous orders displayed
TC10	Update order status	Status updated successfully
TC11	Add menu item	New item added

TC12	Delete menu item	Item removed successfully
TC13	Invalid login	Appropriate error displayed
TC14	Empty cart checkout	Checkout prevented
TC15	Backend API test	Correct API response received

5.2 Testing Procedure

1. Start the application.
2. Open the frontend.
3. Register a new user.
4. Login using valid credentials.
5. Browse restaurants.
6. Search and filter food items.
7. Add food items to cart.
8. Modify quantities.
9. Place an order.
10. Verify order details.
11. Login as restaurant administrator.
12. Update order status.
13. Verify that the customer receives the updated status.
14. Test invalid inputs.
15. Verify backend API responses.

5.3 Testing Evidence

The report should include screenshots of:

- Login page
- Restaurant listing
- Food menu
- Search/filter operation
- Shopping cart
- Order confirmation
- Order tracking
- Admin dashboard
- Backend API response
- Database records
- Docker containers

6. Repository Documentation

6.1 README Structure

The README.md file should contain:

Project Title

Project Description

Features

Technology Stack

System Requirements

Installation

Configuration

Running the Application

Docker Instructions

API Documentation

Testing

Screenshots

Project Structure

Contributors

Future Enhancements

6.2 Installation Instructions

Example:

```
git clone <repository-url>
```

```
cd scalable-food-ordering-system
```

```
cd frontend
```

```
npm install
```

```
cd ../backend
```

```
npm install
```

The application can then be started using the appropriate frontend and backend commands.

6.3 Docker Execution

The complete application can be started using:

```
docker compose up --build
```

Running containers can be checked using:

```
docker ps
```

The application can be stopped using:

```
docker compose down
```

6.4 Documentation Evaluation

Good documentation helps new developers understand:

- Project purpose
- Installation procedure
- Dependencies
- Configuration
- API usage
- Docker deployment
- Testing procedure

The README should be updated whenever major project features are added.

7. Code Review Findings and Recommendations

7.1 Code Review Findings

The review identifies the following positive aspects:

1. The project follows a modular architecture.
2. Frontend and backend responsibilities are separated.
3. REST APIs are used for communication.
4. Git provides proper source-code management.
5. Feature branches support parallel development.
6. Docker provides a consistent deployment environment.
7. Database operations are separated from UI logic.
8. The repository structure is easy to understand.

7.2 Identified Issues

Potential issues include:

- Limited automated testing.
- API response optimization may be required.
- Large food-image files may affect performance.
- Database queries may require optimization.
- Authentication should be strengthened.
- Environment configuration must be handled securely.
- Error logging should be improved.

7.3 Recommendations

The following improvements are recommended:

1. Automated Testing

Introduce unit, integration, and API testing.

2. Caching

Use caching for frequently requested restaurant and menu data.

3. Database Optimization

Create indexes for frequently searched fields such as:

restaurantName

foodName

category

price

rating

orderStatus

4. Authentication

Implement secure authentication using JWT and role-based access control.

5. Container Scaling

Deploy multiple backend containers using Kubernetes when traffic increases.

6. CI/CD

Integrate GitHub with a CI/CD pipeline to automatically:

Commit

↓

Build

↓

Test

↓

Docker Image

↓

Deploy

7. Monitoring

Add application and container monitoring to identify performance problems.



System Requirement Analysis

Hardware Requirements

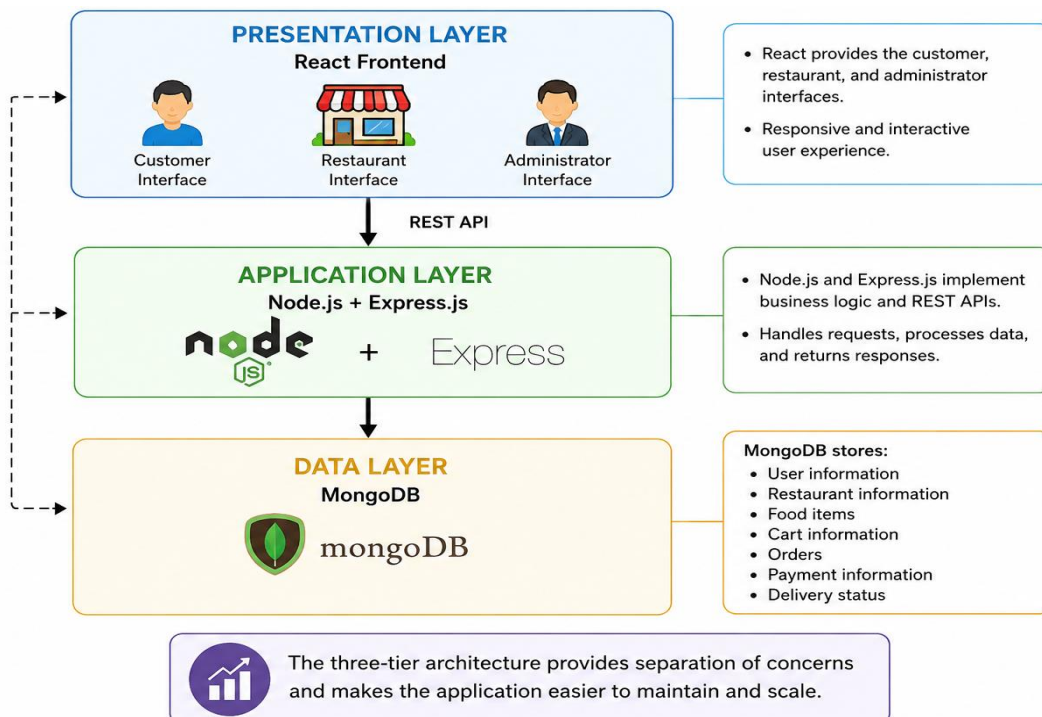
- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Internet connection
- Network-enabled system

Software Requirements

- Windows/Linux/macOS
- Visual Studio Code
- Node.js
- npm
- React
- Node.js
- Express.js
- MongoDB
- Git
- GitHub
- Docker Desktop
- Docker Compose
- Modern web browser

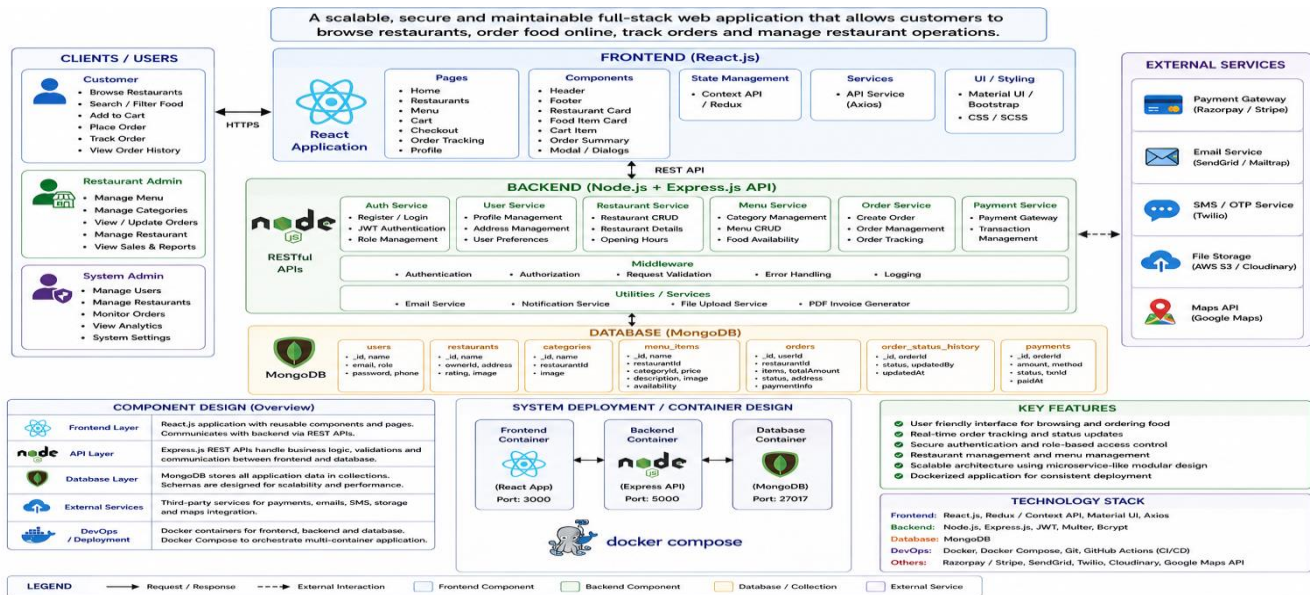
Selection of Software Architecture

A three-tier architecture is selected for the Scalable Online Food Ordering System.



The three-tier architecture provides separation of concerns and makes the application easier to maintain and scale.

Architecture Diagram and Component Design



Quality Attribute Analysis

Quality Attribute Implementation

Performance	Efficient search, filtering and database queries
Scalability	Docker containers and future Kubernetes deployment
Maintainability	Modular frontend and backend architecture
Portability	Docker-based deployment
Security	Authentication, authorization and environment variables
Reliability	Validation, error handling and database persistence
Availability	Container restart and scalable backend services
Usability	Simple and responsive user interface
Testability	Modular components and automated test support
Extensibility	Separate modules for users, restaurants, menus and orders

Architectural Justification

The three-tier architecture is appropriate for the online food ordering system because it separates the user interface, application logic, and data management.

React provides an interactive and responsive frontend. Node.js and Express.js provide lightweight REST APIs for handling users, restaurants, menus, carts, and orders. MongoDB provides flexible storage for application data.

Docker allows the frontend, backend, and database to operate in isolated and reproducible environments. Docker Compose simplifies the management of these services.

Git and GitHub provide version control, branching, collaboration, and code review facilities.

The architecture can be extended in the future using:

- Kubernetes
- Helm
- CI/CD
- Load balancing
- Redis caching
- Message queues
- Cloud deployment
- Microservices

Therefore, the architecture is suitable for a scalable online food ordering application.

Overall Code Review Summary

The Scalable Online Food Ordering System demonstrates the major concepts associated with CO3 by integrating full-stack development, version control, collaborative development, containerization, and scalable architecture.

The React frontend provides the user interface, while Node.js and Express.js implement backend services. MongoDB provides persistent data storage. Git and GitHub support collaborative development through branches, commits, pull requests, and code reviews. Docker provides consistent and portable execution environments.

The proposed improvements, including automated testing, caching, database optimization, CI/CD, Kubernetes deployment, and monitoring, can further improve the system's performance, scalability, reliability, and maintainability.